

std::basic_fixed_string

Document #: P3094R5
Date: 2024-10-15
Project: Programming Language C++
Audience: SG16 Unicode
Library Evolution Working Group
Reply-to: Mateusz Pusz ([Epam Systems](https://www.epam.com))
<mateusz.pusz@gmail.com>

Contents

- 1 Revision history
 - 1.1 Changes since [P3094R4?]
 - 1.2 Changes since [P3094R3]
 - 1.3 Changes since [P3094R2]
 - 1.4 Changes since [P3094R1]
 - 1.5 Changes since [P3094R0]
- 2 Introduction
- 3 fixed_string is an established practice
- 4 Minimal interface requirements
- 5 fixed_string design alternatives
 - 5.1 Full string_view-like interface
 - 5.2 Mutation interface
 - 5.3 inplace_string
 - 5.4 std::string with a static storage allocator
 - 5.5 Using std::array for storage
 - 5.6 Just wait for the C++ language to solve it
- 6 Design discussion
 - 6.1 No default constructor
 - 6.2 Constructor from the string literal
 - 6.3 Constructor taking the list of characters

- 6.4 Constructor from the range
- 6.5 Assignment and swap
- 6.6 `simd-like` size
- 6.7 Non-member swap
- 6.8 User-defined literals
- 6.9 Structural type requirements
- 6.10 `char_traits`
- 7 Open questions
 - 7.1 Should we unhide our friends?
- 8 Implementation experience
- 9 Wording
 - 9.1 Library introduction
 - 9.1.1 Library-wide requirements
 - 9.2 Header `<version>` synopsis
 - 9.3 General utilities library
 - 9.3.1 Formatting
 - 9.4 Strings library
 - 9.4.1 General
 - 9.4.2 String view classes
 - 9.4.3 Fixed size string classes
 - 9.5 Containers library
 - 9.5.1 Requirements
 - 9.6 Annex C (informative)
 - 9.6.1 C++ and ISO C++ 2023
- 10 Acknowledgements
- 11 References

§ 1 Revision history

§ 1.1 Changes since [P3094R4?]

- Footnote modified to express immutability.
- `char_traits` support removed and `char_traits` chapter added.

§ 1.2 Changes since [P3094R3]

- `std::` prefix dropped in [Wording](#).
- [Iterator support](#) and [Element access](#) chapters added.
- one-of exposition-only concept removed.
- Extraneous `const` removed from `operator+`.

§ 1.3 Changes since [P3094R2]

- Additional motivation for [Constructor taking the list of characters](#) added.
- Capacity related operations fixed to properly use `std::integral_constant`.
- `<fixed_string>` added to [Table 24: C++ library headers](#).
- Non-member `swap()` and rationale for it added.
- Ill-formed notes in synopsis replaced with “Mandates”.
- “Constructs an object” replaced with the “Initializes `data_`” in the constructor’s wording.
- “Annex C” entry added.

§ 1.4 Changes since [P3094R1]

- [Design discussion](#) chapter extended.
- [Open questions](#) chapter updated.
- Compiler Explorer link added to the [Implementation experience](#) chapter.
- “Synopsis” chapter renamed to [Wording](#) and filled with standardeese.
- Char traits template parameter added.
- `reverse_iterator` support.
- Functions taking string literals as parameters made `constexpr`.
- Constructor from a range added.
- `view()` member function.
- More container-like and string-like member functions.
- Additional `operator+` overloads that explicitly work with string literals and single characters.
- Comparison operators with string literals.
- Deduction guide from `array<charT, N>`.
- `formatter` specialization removed from synopsis as the `fixed_string` type was added to the debug-enabled string type specializations.

§ 1.5 Changes since [P3094R0]

- [Using `std::array` for storage](#) chapter added.
- [Design rationale] chapter added.
- [Open questions](#) chapter added.
- [Synopsis] chapter updated:
 - `.view()` replaced with a conversion operator to `std::string_view`,
 - constructor taking `std::integral_constant` replaced with the one that takes iterator and sentinel,
 - `operator==` specification improved,
 - missing alias templates for `wchar_t`, `char8_t`, `char16_t`, and `char32_t` added.

§ 2 Introduction

This paper proposes standardizing a string type that could be used at compile-time as a non-type template parameter (NTTP). This means that such string type has to satisfy the structural type requirements of the C++ language. One of such requirements is to expose all the data members publicly. So far, none of the existing string types in the C++ standard library satisfies such requirements.

This type is intended to serve as an essential building block and be exposed as a part of the public interface of quantities and units library proposed in [P2980R1]. All dimensions, units, prefixes, and constants definitions will use it to provide their textual representation.

§ 3 `fixed_string` is an established practice

`fixed_string` is not just to enable quantities and units library. There are plenty of home-grown `fixed_string` types in the wild already. It is heavily used in low-latency finance, text formatting, compile-time regular expressions, and many other domains.

Today, every project that wants to pass text as NTTP already provides its own version of such type. There are also some that predate C++20 and do not satisfy structural type requirements.

A [quick search in GitHub](#) returns plenty of results. Let's mention a few of those:

- [mpusz/mp-units](#)
- [fmtlib/fmt](#)
- [hanickadot/compile-time-regular-expressions](#)
- [tomazos/fixed_string](#) - a reference implementation of [P0259R0] (not a structural type as it is 8 years old)

- [unterumarmung/fixed_string](#)
- [mu001999/fixed_string](#)
- [calebxyz/Fixed_Strings](#)
- [astral-shining/FixedString](#)
- [ynsn/fixed_string](#)

Having such a type in the C++ standard library will prevent reinventing the wheel by the community and reimplementing it in every project that handles text at compile-time.

§ 4 Minimal interface requirements

Such type should at least:

- satisfy structural type requirements,
- be equality comparable and potentially totally ordered,
- store and provide concatenation support for zero-ended strings,
- provide storage that, if set at compile time, would also be available for read-only access at runtime,
- provide at least read-only access to the contained storage.

It does not need to expose a string-like interface. In case its interface is immutable, as it is proposed in this paper, we can easily wrap it with `std::string_view` to get such an interface for free.

§ 5 `fixed_string` design alternatives

§ 5.1 Full `string_view`-like interface

We could add a whole `string_view`-like interface to this class, but the author decided not to do it. This will add plenty of overloads that will probably not be used too often by the users of this particular type anyway. Doing that will also add a maintenance burden to keep it consistent with all other string-like types in the library.

If the user needs to obtain a `string_view`-like interface to work with this type, then `std::string_view` itself can easily be used to achieve that:

```
std::basic_fixed_string txt = "abc";  
auto pos = txt.view().find_first_of('b');
```

§ 5.2 Mutation interface

As stated above, this type is read-only. As we can't resize the string, it could be considered controversial to allow mutation of the contents. The only thing we could provide would be support for overwriting specific characters in the internal storage. This, however, is not a common use case for such a type.

If passed as an NTTP, such type would be `const` at runtime, which would disable all the mutation interface anyway.

On the other hand, such an interface would allow running every `constexpr` algorithm from the C++ standard library on such a range at compile time.

Suppose we decide to add such an interface. In that case, it is worth pointing out that wrapping the type in the `std::string_view` would not be enough to obtain a proper string-like mutating interface. As we do not have another string-like reference type that provides mutating capabilities, we can end up with the need to implement the entire string interface in this class as well.

This is why the author does not propose adding it to the first iteration. We can always easily add such an interface later if a need arises.

§ 5.3 `inplace_string`

We could also consider providing a full-blown fixed-capacity string class similar to the `inplace_vector` [P0843R9]. It would provide not only full read-write capability but also be really beneficial for embedded and low-latency domains.

Despite being welcomed and valuable in the C++ community, the author believes that such a type should not satisfy the current requirements for a structural type, which is a hard requirement of this proposal. If `inplace_string` was used instead, we would end up with separate template instantiations for objects with the same value but a different capacity. This is why we should not consider adding such a type to the library for now.

§ 5.4 `std::string` with a static storage allocator

We could also try to use `std::string` with a static storage allocator, but this solution does not meet structural type requirements and could be an overkill for this use case.

Also, it would be hard or impossible to make the storage created at compile-time to be accessible at runtime in such cases.

§ 5.5 Using `std::array` for storage

We could consider using `std::array` instead, as it satisfies the current requirements for structural types. However, it does not properly construct from string literals and does not provide string-like concatenation interfaces.

§ 5.6 Just wait for the C++ language to solve it

[P2484R0] proposed extending support for class types as non-type template parameters in the C++ language. However, this proposal’s primary author is no longer active in C++ standardization, and there have been no updates to the paper in the last two years.

We can’t wait for the C++ language to change forever. For example, the quantities and units library will be impossible to standardize without such a feature. This is why the author recommends progressing with the `basic_fixed_string` approach.

§ 6 Design discussion

§ 6.1 No default constructor

As this type has sized fixed at compile time it can’t be empty upon construction. This means that this type can’t provide the default constructor. This also means that it does not satisfy the constraints of the `std::regular` concept.

§ 6.2 Constructor from the string literal

This constructor is the workhorse of `fixed_string` type. It will be called by the users every time they want to pass string literals to NTTPs. This is why the author proposes to make it implicit. Making the constructors explicit would make the primary use case much more verbose:

Explicit	Implicit
<pre>inline constexpr struct second : named_unit<std::basic_fixed_string{"s"}> {} second; inline constexpr struct metre : named_unit<std::fixed_string{"m"}> {} metre;</pre>	<pre>inline constexpr struct second : named_unit<"s"> {} second; inline constexpr struct metre : named_unit<"m"> {} metre;</pre>

Typically, we agree on making the constructor implicit if:

1. The constructor argument and the type itself represent “the same” abstraction.
2. There is no significant overhead of calling such a constructor.

Point 1. is satisfied as both a string literal and `basic_fixed_string` represent an array of characters ended with `\0`.

Additionally, the author proposes to make this constructor `constexpr`. This should encourage calling it with string literals that by definition end with `\0` character and limit chances of passing a hand-made array of characters. This approach got unanimous consent after discussion in LEWGI in Tokyo 2024. This approach also extends to all other functions taking the array of `charT` as parameters to improve consistency and correctness.

As calling such an immediate function does not impose any runtime overhead, the construction of this type also satisfies point 2. above.

§ 6.3 Constructor taking the list of characters

This constructor enables a few use cases that are hard to implement otherwise:

1. `std::basic_fixed_string(static_cast<char>('0' + Value))` (used in `mp-units`)
2. `std::fixed_string{'V', 'P', 'B', (char)version}` (credit to Hana Dusíková)
3. `std::fixed_string{'W', 'e', 'i', 'r', 'd', '\\0', 'b', 'u', 't', ' ', 't', 'r', 'u', 'e'}` (credit to Tom Honermann)

This constructor also allows us to deduce the size of the string from the initializer to improve CTAD usage. Using `std::initializer_list<charT>` would not support it.

§ 6.4 Constructor from the range

There are at least two ways of passing a range to a string-like type in the C++ standard library:

- `basic_string` takes `from_range_t` as the first parameter and `R&&` as the second one,
- `basic_string_view` takes `R&&` as the only parameter.

If we decided to go with the latter case, this constructor would hijack the constructor taking the array of characters as a parameter. This is why we would need to provide a bit different signature than `string_view`. Taking `const R&` parameter would be good enough as move on character types is not faster than a copy.

Moreover, this type owns its storage, so it is more similar to `basic_string` than the `basic_string_view`.

This is why the author decided to propose the first alternative that uses `from_range_t` as an additional first parameter.

§ 6.5 Assignment and swap

Even though the type does not provide any string/container-like interface to mutate the contents, they can still be changed with the copy assignment operator. Not exposing this operator could be surprising to many users. Also, as we have assignment probably it is good to also provide a `swap()` member function for consistency with other types in the library.

§ 6.6 simd-like size

During Tokyo 2024 discussion in LEWGI we took the following poll:

POLL: Do we want the `.size` member to be an `integral_constant<size_t, N>` (and `.empty` to be `bool_constant<N==0>`)?

SF	WF	N	WA	SA
5	2	2	2	0

As a result, the R2 version of this paper introduces this practice from the `simd` type to `size`, `length`, `max_size`, and `empty` member functions.

§ 6.7 Non-member swap

Typically its implementation will not differ from calling the generic `std::swap`. However, in some extreme cases, when we use the type with enormously long text, library implementers may choose to swap the data in chunks to save stack space. In such case, generic `std::swap()` could provide a different behavior than calling the member `swap()` function.

§ 6.8 User-defined literals

In Tokyo 2024 we briefly discussed adding UDLs for this type. However, we decided not to do it as those strings are not massively used and typically are not created on the fly from literals in user's code.

As there is a dedicated constructor that takes a string literal as an argument, this is a recommended way to initialize the object of this type from a string literal.

§ 6.9 Structural type requirements

As of today, the C++ language requires a structural type to expose all its members publicly. This is why in the wording a public exposition-only data member is provided. However, we hope that the structural type requirements will be relaxed in the future and this data member will be possible to implement as a private data member in the future versions of the standard.

§ 6.10 char_traits

Support for `char_traits` was added upon the LEWGI request in [P3094R1]. However, this was considered a bad decision by SG16.

`char_traits` promises configurability for operations where no user-level configuration makes sense.

The `charT` template parameter needs to be a char-like type (23.1 [strings.general]), which means it's trivial, and the implementation thus doesn't need `char_traits` to enable memcpy-style optimizations.

For the comparisons, having `char_traits` that implement case-insensitive comparisons is oversimplifying because case-insensitivity is locale-dependent. Also, encoding such comparison semantics into a data type seems like a bad design.

During the SG16 meeting on October 9, 2024, the following poll was taken:

POLL: P3094R4: Amend the proposal to remove the `traits` template parameter and dependence on `std::char_traits`.

SF	WF	N	WA	SA
4	5	1	0	0

- Attendees: 12 (2 abstentions)
- Author's position: N
- Strong consensus

As a result of this poll, `char_traits` were removed again from the paper in R5.

§ 7 Open questions

§ 7.1 Should we unhide our friends?

A modern practice is to expose operators as hidden friends and this is what this type does. However, this is inconsistent with `string` and `string_view` which provide them as regular non-member functions. Should we consider doing the same for consistency?

§ 8 Implementation experience

This particular interface is implemented, tested, and successfully used in the [mp-units](#) project. A complete implementation with tests can also be checked in the [Compiler Explorer](#).

§ 9 Wording

§ 9.1 Library introduction

§ 9.1.1 Library-wide requirements

§ 9.1.1.1 Library contents and organization

9.1.1.1.1 Headers

Add `<fixed_string>` to the [Table 24: C++ library headers](#).

§ 9.2 Header `<version>` synopsis

```
#define __cpp_lib_filesystem 201703L // also in
    <filesystem>
#define __cpp_lib_fixed_string YYYYMMML // also in
    <fixed_string>
#define __cpp_lib_flat_map 202207L // also in
    <flat_map>
```

Editorial note: Adjust the placeholder value as needed so as to denote this proposal’s date of adoption.

§ 9.3 General utilities library

§ 9.3.1 Formatting

§ 9.3.1.1 Formatter

9.3.1.1.1 Formatter specializations

(2.2) For each charT, the debug-enabled string type specializations

```
template<> struct formatter<charT*, charT>;
template<> struct formatter<const charT*, charT>;
template<size_t N> struct formatter<charT[N], charT>;
template<class traits, class Allocator>
    struct formatter<basic_string<charT, Allocator>, charT>;
template<class traits>
    struct formatter<basic_string_view<charT>, charT>;
template<size_t N>
    struct formatter<basic_fixed_string<charT, N>, charT>;
```

§ 9.4 Strings library

§ 9.4.1 General

Modify Table 78: Strings library summary:

	Subclause	Header
23.2 [char.traits]	Character traits	<string>
23.3 [string.view]	String view classes	<string_view>
23.4 [string.classes]	String classes	<string>
[fixed.string]	Fixed size string classes	<fixed_string>
23.5 [c.strings]	Null-terminated sequence utilities	<cctype>, <cstdlib>, <cstring>, <cuchar>, <wchar>, <cwctype>

§ 9.4.2 String view classes

§ 9.4.2.1 General

- 2 [Note 1: The library provides implicit conversions from `const charT*` ~~and~~, `std::basic_string<charT, ...>`, and `std::basic_fixed_string<charT, ...>` to `std::basic_string_view<charT, ...>` so that user code can accept just `std::basic_string_view<charT>` as a non-templated parameter wherever a sequence of characters is expected. User-defined types can define their own implicit conversions to `std::basic_string_view<charT>` in order to interoperate with these functions. — *end note*]

§ 9.4.3 Fixed size string classes

§ 9.4.3.1 General

- 1 The header `<fixed_string>` defines the `basic_fixed_string` class template that stores a read-only fixed-length sequences of char-like objects and five [typedef-names](#), `fixed_string`, `fixed_u8string`, `fixed_u16string`, `fixed_u32string`, and `fixed_wstring`, that name the specializations `basic_fixed_string<char>`, `basic_fixed_string<char8_t>`, `basic_fixed_string<char16_t>`, `basic_fixed_string<char32_t>`, and `basic_fixed_string<wchar_t>`, respectively.

§ 9.4.3.2 Header `<fixed_string>` synopsis

```
// mostly freestanding
#include <compare>           // see [compare.syn]
#include <string_view>       // see [string.view.synop]

namespace std {

// [fixed.string.template], class template basic_fixed_string
template<class charT, size_t N>
class basic_fixed_string; // partially freestanding

// [fixed.string.special], specialized algorithms
template<class charT, size_t N>
constexpr void swap(basic_fixed_string<charT, N>& x,
                    basic_fixed_string<charT, N>& y) noexcept;

// basic_fixed_string typedef-names
template<size_t N> using fixed_string    = basic_fixed_string<char, N>;
template<size_t N> using fixed_u8string  = basic_fixed_string<char8_t, N>;
template<size_t N> using fixed_u16string = basic_fixed_string<char16_t, N>;
template<size_t N> using fixed_u32string = basic_fixed_string<char32_t, N>;
template<size_t N> using fixed_wstring   = basic_fixed_string<wchar_t, N>;

// [fixed.string.hash], hash support
template<size_t N> struct hash<fixed_string<N>> : hash<string_view> {};
template<size_t N> struct hash<fixed_u8string<N>> : hash<u8string_view> {};
};
```

```

template<size_t N> struct hash<fixed_u16string<N>> : hash<u16string_view>
    {};
template<size_t N> struct hash<fixed_u32string<N>> : hash<u32string_view>
    {};
template<size_t N> struct hash<fixed_wstring<N>> : hash<wstring_view>
    {};
}

```

- 1 The function templates defined in [utility.swap](#) and [iterator.range](#) are available when `<fixed_string>` is included.

§ 9.4.3.3 Class template `basic_fixed_string`

- 1 A specialization of `basic_fixed_string` is a [contiguous container](#) storing a fixed-size sequence of characters. An instance of `basic_fixed_string<charT, N>` stores `N` elements of type `charT`, so that `size() == N` is an invariant.
- 2 A `fixed_string` meets all of the requirements of a container (24.2.2.2 [\[container.reqmts\]](#)) and of a reversible container (24.2.2.3 [\[container.rev.reqmts\]](#)), except that `fixed_string` can't be default constructed. A `fixed_string` meets some of the requirements of a sequence container (24.2.4 [\[sequence.reqmts\]](#)). Descriptions are provided here only for operations on `fixed_string` that are not described in one of these tables and for operations where there is additional semantic information.
- 3 `basic_fixed_string<charT, N>` is a structural type (13.2 [\[temp.param\]](#)) if `charT` is structural types. Two values `fs1` and `fs2` of type `basic_fixed_string<charT, N>` are template-argument-equivalent if and only if each pair of corresponding elements in `fs1` and `fs2` are template-argument-equivalent.
- 4 In all cases, `[data(), data() + size())` is a valid range and `data() + size()` points at an object with value `charT()` (a “null terminator”).

9.4.3.3.1 General

```

namespace std {

template<class charT, size_t N>
class basic_fixed_string {
public:
    charT data_[N + 1] = {}; // exposition only

    // types
    using value_type      = charT;
    using pointer         = value_type*;
    using const_pointer   = const value_type*;
    using reference       = value_type&;
    using const_reference = const value_type&;
    using const_iterator  = implementation-defined; // see
        fixed.string.iterators
    using iterator        = const_iterator; [^1]
    using const_reverse_iterator = reverse_iterator<const_iterator>;

```

```

using reverse_iterator      = const_reverse_iterator;
using size_type             = size_t;
using difference_type       = ptrdiff_t;

// [fixed.string.cons], construction and assignment
template<convertible_to<charT>... Chars>
    requires(sizeof...(Chars) == N) && (... && !is_pointer_v<Chars>)
constexpr explicit basic_fixed_string(Chars... chars) noexcept;

constexpr basic_fixed_string(const charT (&txt)[N + 1]) noexcept;

template<input_iterator It, sentinel_for<It> S>
    requires convertible_to<iter_value_t<It>, charT>
constexpr basic_fixed_string(It begin, S end);

template<container-compatible-range<charT> R>
constexpr basic_fixed_string(from_range_t, R&& r);

    constexpr basic_fixed_string(const basic_fixed_string&) noexcept =
        default;
    constexpr basic_fixed_string& operator=(const basic_fixed_string&)
        noexcept = default;

// [fixed.string.iterators], iterator support
constexpr const_iterator begin() const noexcept;
constexpr const_iterator end() const noexcept;
constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr const_reverse_iterator rend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// capacity
static constexpr integral_constant<size_type, N> size{};
static constexpr integral_constant<size_type, N> length{};
static constexpr integral_constant<size_type, N> max_size{};
static constexpr bool_constant<N == 0> empty{};

// [fixed.string.access], element access
constexpr const_reference operator[](size_type pos) const;
constexpr const_reference at(size_type pos) const; // freestanding-
    deleted
constexpr const_reference front() const;
constexpr const_reference back() const;

// [fixed.string.modifiers], modifiers
constexpr void swap(basic_fixed_string& s) noexcept;

// [fixed.string.ops], string operations
constexpr const_pointer c_str() const noexcept;
constexpr const_pointer data() const noexcept;
constexpr basic_string_view<charT> view() const noexcept;
constexpr operator basic_string_view<charT>() const noexcept;
template<size_t N2>

```

```

constexpr friend basic_fixed_string<charT, N + N2> operator+(const
    basic_fixed_string& lhs,
                                                                    const
    basic_fixed_string<charT, N2>& rhs) noexcept;
constexpr friend basic_fixed_string<charT, N + 1> operator+(const
    basic_fixed_string& lhs, charT rhs) noexcept;
constexpr friend basic_fixed_string<charT, 1 + N> operator+(const charT
    lhs, const basic_fixed_string& rhs) noexcept;
template<size_t N2>
constexpr friend basic_fixed_string<charT, N + N2 - 1> operator+(const
    basic_fixed_string& lhs,
                                                                    const
    charT (&rhs)[N2]) noexcept;
template<size_t N1>
constexpr friend basic_fixed_string<charT, N1 + N - 1> operator+(const
    charT (&lhs)[N1],
                                                                    const
    basic_fixed_string& rhs) noexcept;

// [fixed.string.comparison], non-member comparison functions
template<size_t N2>
    friend constexpr bool operator==(const basic_fixed_string& lhs, const
        basic_fixed_string<charT, N2>& rhs);
template<size_t N2>
    friend constexpr bool operator==(const basic_fixed_string& lhs, const
        charT (&rhs)[N2]);

template<size_t N2>
    friend constexpr see below operator<=(const basic_fixed_string& lhs,
        const basic_fixed_string<charT, N2>& rhs);
template<size_t N2>
    friend constexpr see below operator<=(const basic_fixed_string& lhs,
        const charT (&rhs)[N2]);

// [fixed.string.io], inserters and extractors
friend basic_ostream<charT>& operator<<(basic_ostream<charT>& os, const
    basic_fixed_string& str); // hosted
};

// [fixed.string.deduct], deduction guides
template<typename CharT, convertible_to<CharT>... Rest>
    basic_fixed_string(CharT, Rest...) -> basic_fixed_string<CharT, 1 +
        sizeof...(Rest)>;

template<typename charT, size_t N>
    basic_fixed_string(const charT (&str)[N]) -> basic_fixed_string<charT, N -
        1>;

template<typename CharT, size_t N>
    basic_fixed_string(from_range_t, array<CharT, N>) ->
        basic_fixed_string<CharT, N>;
}

```

Footnote ^[1]: Because `basic_fixed_string` is immutable, `iterator` and `const_iterator` are the same type.

§ 9.4.3.4 Construction and assignment

```
template<convertible_to<charT>... Chars>
    requires(sizeof...(Chars) == N) && (... && !is_pointer_v<Chars>)
constexpr explicit basic_fixed_string(Chars... chars) noexcept;
```

- 1 *Effects:* Initializes `data_` with the values of `chars...` before the call followed by the `charT()` (a “null terminator”).

```
constexpr basic_fixed_string(const charT (&txt)[N + 1]);
```

- 2 *Mandates:* `txt[N] == CharT()`.

- 3 *Effects:* Initializes `data_` with a copy of `txt`.

```
template<input_iterator It, sentinel_for<It> S>
    requires convertible_to<iter_value_t<It>, charT>
constexpr basic_fixed_string(It begin, S end);
```

- 4 *Preconditions:* `distance(begin, end) == N`.

- 5 *Effects:* Initializes `data_` from the values in the range `[begin, end)`, as specified in 24.2.4 [sequence.reqmts].

```
template<container-compatible-range<charT> R>
constexpr basic_fixed_string(from_range_t, R&& r);
```

- 6 *Preconditions:* `ranges::size(r) == N`.

- 7 *Effects:* Initializes `data_` from the values in the range `rg`, as specified in 24.2.4 [sequence.reqmts].

§ 9.4.3.5 Iterator support

```
using const_iterator = implementation-defined;
```

- 1 A type that meets the requirements of a constant Cpp17RandomAccessIterator 25.3.5.7 [random.access.iterators], models contiguous_iterator 25.3.4.14 [iterator.concept.contiguous], and meets the `constexpr` iterator requirements 25.3.1 [iterator.requirements.general], whose `value_type` is the template parameter `charT`.

- 2 All requirements on container iterators 24.2 [container.requirements] apply to `fixed_string::const_iterator` as well.

```
constexpr const_iterator begin() const noexcept;
constexpr const_iterator cbegin() const noexcept;
```

- 3 *Returns:* An iterator referring to the first character in the string.

```
constexpr const_iterator end() const noexcept;
constexpr const_iterator cend() const noexcept;
```


4 *Returns:* An iterator which is the past-the-end value.

```
constexpr const_reverse_iterator rbegin() const noexcept;  
constexpr const_reverse_iterator crbegin() const noexcept;
```

5 *Returns:* `const_reverse_iterator(end())`.

```
constexpr const_reverse_iterator rend() const noexcept;  
constexpr const_reverse_iterator crend() const noexcept;
```

6 *Returns:* `const_reverse_iterator(begin())`.

§ 9.4.3.6 Element access

```
constexpr const_reference operator[](size_type pos) const;
```

1 *Preconditions:* `pos < size()`.

2 *Returns:* `data_[pos]`.

3 *Throws:* Nothing.

4 [Note 1: Unlike `basic_string::operator[]`, `basic_fixed_string::operator[]` (`size()`) has undefined behavior instead of returning `charT()`. — end note]

```
constexpr const_reference at(size_type pos) const;
```

5 *Returns:* `data_[pos]`.

6 *Throws:* `out_of_range` if `pos >= size()`.

```
constexpr const_reference front() const;
```

7 *Preconditions:* `!empty()`.

8 *Returns:* `data_[0]`.

9 *Throws:* Nothing.

```
constexpr const_reference back() const;
```

10 *Preconditions:* `!empty()`.

11 *Returns:* `data_[size() - 1]`.

12 *Throws:* Nothing.

§ 9.4.3.7 Modifiers

```
constexpr void swap(basic_fixed_string& s) noexcept;
```

1 *Effects:* Equivalent to `swap_ranges(begin(), end(), s.begin())`.

- 2 [Note 1: Unlike the swap function for other containers, `basic_fixed_string::swap` takes linear time and does not cause iterators to become associated with the other container. — end note]

§ 9.4.3.8 String operations

```
constexpr const_pointer c_str() const noexcept;  
constexpr const_pointer data() const noexcept;
```

- 1 *Returns:* A pointer `p` such that `p + i == addressof(operator[](i))` for each `i` in `[0, size())`.
- 2 *Complexity:* Constant time.
- 3 *Remarks:* The program shall not modify any of the values stored in the character array; otherwise, the behavior is undefined.

```
constexpr basic_string_view<charT> view() const noexcept;  
constexpr operator basic_string_view<charT>() const noexcept;
```

- 4 *Effects:* Equivalent to: `return basic_string_view<charT>(data(), size());`

```
template<size_t N2>  
constexpr friend basic_fixed_string<charT, N + N2> operator+(const  
    basic_fixed_string& lhs,                                     const  
    basic_fixed_string<charT, N2>& rhs) noexcept;  
constexpr friend basic_fixed_string<charT, N + 1> operator+(const  
    basic_fixed_string& lhs, charT rhs) noexcept;  
constexpr friend basic_fixed_string<charT, 1 + N> operator+(charT lhs,  
    const basic_fixed_string& rhs) noexcept;
```

- 5 *Effects:* Returns a new fixed string object whose value is the joint value of `lhs` and `rhs` followed by the `charT()`.

```
template<size_t N2>  
constexpr friend basic_fixed_string<charT, N + N2 - 1> operator+(const  
    basic_fixed_string& lhs,                                     const  
    charT (&rhs)[N2]) noexcept;
```

- 6 *Mandates:* `rhs[N2 - 1] == CharT()`.
- 7 *Effects:* Returns a new fixed string object whose value is the joint value of `lhs` and `rhs`.

```
template<size_t N1>  
constexpr friend basic_fixed_string<charT, N1 + N - 1> operator+(const  
    charT (&lhs)[N1],                                         const  
    basic_fixed_string& rhs) noexcept;
```

- 8 *Mandates:* `lhs[N1 - 1] == CharT()`.

- 9 *Effects*: Returns a new fixed string object whose value is the joint value of a subrange `[lhs[0], lhs[N1 - 1])` and rhs followed by the `charT()`.

§ 9.4.3.9 Non-member comparison functions

```
template<size_t N2>
friend constexpr bool operator==(const basic_fixed_string& lhs, const
    basic_fixed_string<charT, N2>& rhs);
template<size_t N2>
friend constexpr see below operator<==(const basic_fixed_string& lhs,
    const basic_fixed_string<charT, N2>& rhs);
```

- 1 *Effects*: Let op be the operator. Equivalent to:

```
return lhs.view() op rhs.view();
```

```
template<size_t N2>
friend constexpr bool operator==(const basic_fixed_string& lhs, const
    charT (&rhs)[N2]);
template<size_t N2>
friend constexpr see below operator<==(const basic_fixed_string& lhs,
    const charT (&rhs)[N2]);
```

- 2 *Mandates*: `rhs[N2 - 1] == CharT()`.

- 3 *Effects*: Let op be the operator. Equivalent to:

```
return lhs.view() op basic_string_view<CharT>(rhs, rhs + N2 - 1);
```

§ 9.4.3.10 Inserters and extractors

```
friend basic_ostream<charT>& operator<<(basic_ostream<charT>& os, const
    basic_fixed_string& str);
```

- 1 *Effects*: Equivalent to: `return os << str.view();`

§ 9.4.3.11 Hash support

```
template<class charT, size_t N, class traits>
constexpr void swap(basic_fixed_string<charT, N>& x,
    basic_fixed_string<charT, N>& y) noexcept;
```

- 1 *Effects*: As if by `x.swap(y)`.

- 2 *Complexity*: Linear in N.

§ 9.4.3.12 Hash support

```
template<size_t N> struct hash<fixed_string<N>> : hash<string_view> {};
template<size_t N> struct hash<fixed_u8string<N>> : hash<u8string_view>
    {};
template<size_t N> struct hash<fixed_u16string<N>> : hash<u16string_view>
    {};
```

```
template<size_t N> struct hash<fixed_u32string<N>> : hash<u32string_view>
{};
template<size_t N> struct hash<fixed_wstring<N>> : hash<wstring_view>
{};
```

- 1 If FS is one of these fixed string types, SV is the corresponding string view type, and fs is an object of type FS, then `hash<FS>()(fs) == hash<SV>()(SV(fs))`.

§ 9.5 Containers library

§ 9.5.1 Requirements

§ 9.5.1.1 General containers

9.5.1.1.1 Container requirements

```
X u(rv);
X u = rv;
```

- 15 *Postconditions*: u is equal to the value that rv had before this construction.
- 16 *Complexity*: Linear for array and basic fixed string, ~~and~~ constant for all other standard containers.

```
t.swap(s)
```

- 48 *Result*: `void`.
- 49 *Effects*: Exchanges the contents of t and s.
- 50 *Complexity*: Linear for array and basic fixed string, ~~and~~ constant for all other standard containers.
- 65 The expression `a.swap(b)`, for containers a and b of a standard container type other than array and basic fixed string, shall exchange the values of a and b without invoking any move, copy, or swap operations on the individual container elements.

§ 9.5.1.2 Allocator-aware containers

- 1 Except for array and basic fixed string, all of the containers defined in 24 [containers], 19.6.4 [stacktrace.basic], 23.4.3 [basic.string], and 32.9 [re.results] meet the additional requirements of an allocator-aware container, as described below.

§ 9.5.1.3 Sequence containers

- 1 A sequence container organizes a finite set of objects, all of the same type, into a strictly linear arrangement. The library provides four basic kinds of sequence containers: vector, forward_list, list, and deque. In addition, array and basic fixed string are ~~is~~ provided as ~~a~~ sequence containers which provides limited sequence operations because ~~it~~ ~~has~~ they have a fixed number of elements. The library also provides container adaptors

that make it easy to construct abstract data types, such as stacks, queues, flat_maps, flat_multimaps, flat_sets, or flat_multisets, out of the basic sequence container kinds (or out of other program-defined sequence containers).

```
a.front()
```

69 *Result:* reference; const_reference for constant a.

70 *Returns:* `*a.begin()`

71 *Remarks:* Required for basic_string, [basic_fixed_string](#), array, deque, forward_list, list, and vector.

```
a.back()
```

72 *Result:* reference; const_reference for constant a.

73 *Effects:* Equivalent to:

```
auto tmp = a.end();  
--tmp;  
return *tmp;
```

74 *Remarks:* Required for basic_string, [basic_fixed_string](#), array, deque, list, and vector.

```
a[n]
```

117 *Result:* reference; const_reference for constant a

118 *Effects:* Equivalent to: `return *(a.begin() + n);`

119 *Remarks:* Required for basic_string, [basic_fixed_string](#), array, deque, and vector.

```
a.at(n)
```

120 *Result:* reference; const_reference for constant a

121 *Returns:* `*(a.begin() + n)`

122 *Throws:* out_of_range if `n >= a.size()`.

123 *Remarks:* Required for basic_string, [basic_fixed_string](#), array, deque, and vector.

§ 9.6 Annex C (informative)

§ 9.6.1 C++ and ISO C++ 2023

§ 9.6.1.1 [library]: library introduction

1 **Affected subclause:** [headers]

Change: New headers.

Rationale: New functionality.

Effect on original feature: The following C++ headers are new: `<debugging>`, `<fixed_string>`, `<hazard_pointer>`, `<linalg>`, `<rcu>`, and `<text_encoding>`. Valid C++ 2023 code that `#includes` headers with these names may be invalid in this revision of C++.

§ 10 Acknowledgements

Special thanks and recognition goes to [Epam Systems](#) for supporting Mateusz's membership in the ISO C++ Committee and the production of this proposal.

The author would also like to thank Hana Dusíková and Nevin Liber for providing valuable feedback that helped him shape the final version of this document.

§ 11 References

[P0259R0] Michael Price, Andrew Tomazos. 2016-02-12. `fixed_string`: a compile-time string.

<https://wg21.link/p0259r0>

[P0843R9] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2023-09-14. `inplace_vector`.

<https://wg21.link/p0843r9>

[P2484R0] Richard Smith. 2021-11-17. Extending class types as non-type template parameters.

<https://wg21.link/p2484r0>

[P2980R1] Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Charles Hogg, Nicolas Holthaus, Roth Michaels, Vincent Reverdy. 2023-11-28. A motivation, scope, and plan for a quantities and units library.

<https://wg21.link/p2980r1>

[P3094R0] Mateusz Pusz. 2024-02-05. `std::basic_fixed_string`.

<https://wg21.link/p3094r0>

[P3094R1] Mateusz Pusz. 2024-03-20. `std::basic_fixed_string`.

<https://wg21.link/p3094r1>

[P3094R2] Mateusz Pusz. 2024-05-22. `std::basic_fixed_string`.

<https://wg21.link/p3094r2>

[P3094R3] Mateusz Pusz. 2024-06-30. `std::basic_fixed_string`.

<https://wg21.link/p3094r3>